

# DeepGeoGuessr

Aarhus University - Deep Learning - Group 7

December 11, 2025

**Andreas H. Bugge**   **Daniel M. Pedersen**   **Kristian Frølich**   **Martin H. Bruun**  
 au696494   au695209   au617888   au643169

## I. INTRODUCTION

Determining the geographic location of a random image is a challenging task that requires broad knowledge of global visual features, ranging from road infrastructure and signage to vegetation and architectural styles. This is the objective of the online game GeoGuessr, where players compete to identify the location of images captured throughout the world. With recent advances in deep learning, models for image classification have reached remarkable results on datasets such as ImageNet[1], but image geolocation is a more complex task in many aspects. In this project, we investigate the use of a deep learning approach for image geolocation, using a state-of-the-art network ConvNeXt-Base-384[2] to predict locations from a single Google Street View image.

## II. DATASET

The dataset used in this project contains more than one million images, each associated with geographic coordinates. These images come from street-view panoramas, at each location, four images were captured, one for each cardinal direction. Data collection was performed by scraping internal Google Maps API endpoints that expose metadata and low-resolution thumbnails. These endpoints did not require an API key and did not have request limits, unlike the official Street View API, but were constrained in both resolution and retrieval speed. To illustrate the geographic distribution of the data, a density heatmap of image locations was generated using a  $\sim 5\%$  subset of the complete dataset.

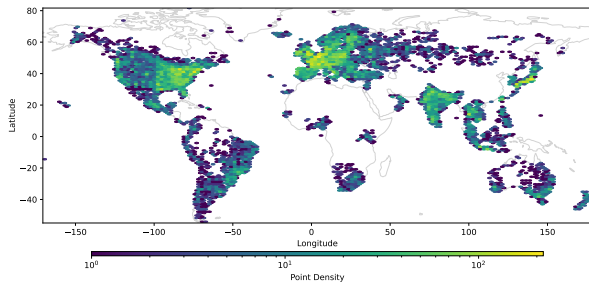


Fig. 1: Heatmap illustrating the geographic distribution of the image dataset. White means no data.

As illustrated in Figure 1, the geographic coverage of the dataset is strongly skewed toward regions with extensive Google Street View coverage. Large parts of the world are either sparsely represented or completely missing. Both the

training and evaluation sets are drawn from this same underlying distribution. The reported evaluation metrics do not reflect performance under a uniform sampling of locations across Earth. The model is evaluated more often in regions it also saw more frequently during training. The performance results as thus inflated for certain areas. For our target application, the game GeoGuessr, this bias is less problematic. The game itself appears to sample locations with a distribution that is similarly concentrated in areas with good Street View coverage<sup>1</sup>. In that sense, the dataset bias is aligned with the intended use case.

TABLE I:  
Statistics for dataset

| Metric         | Value     |
|----------------|-----------|
| Countries      | 115       |
| Excluded zones | 112       |
| Images         | 1,054,108 |
| $L_3$ classes  | 173       |
| $L_4$ classes  | 450       |
| $L_5$ classes  | 1268      |
| $L_6$ classes  | 3740      |
| $L_7$ classes  | 10930     |

Table I contains the statistics for the dataset. A total of 112 countries and zones were excluded, as they are small zones of disputed territory or countries without Street View coverage. This left us with images from 115 countries. The  $L_3$  to  $L_7$  covers the number of classes for different cell sizes used for the hierarchal training approach explained in Section IV.

### A. Dataset Optimization

Given the large volume of images in the dataset (130+ GB), preprocessing and optimization steps were performed to ensure efficient training. All images were resized to a uniform resolution of 384×384 pixels, to ensure consistency. This standardization reduces memory requirements and reduces the spatial dimensions of all feature maps of the model, thus decreasing the number of convolutional operations required at each layer. Hence, improve the overall training efficiency. The dataset was stored as shards (zip-files of 10,000 images), to make it more manageable and reduce the i/o overhead that comes with so many individual files. It was stored directly on AU's GPU Cluster master node (separately from the compute nodes). The shards could then be conveniently and efficiently streamed for computation using the WebDataset package.

<sup>1</sup>Community empirical estimates show similarly skewed regional frequencies. [3]

### III. NETWORK STRUCTURE

The architecture is based on ConvNeXt[2], with the selected model being the ConvNeXt-Base-384 variant, which was pre-trained on ImageNet-1K at an input resolution of  $384 \times 384$  pixels and containing approximately 88 million parameters.

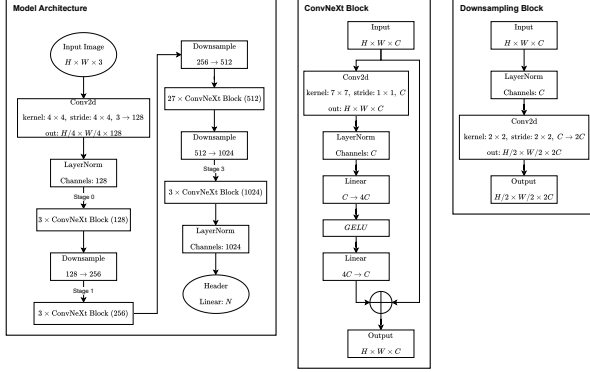


Fig. 2: Model Architecture Flow diagram.  $N$  represent the number of Output classes.

The model begins with path embedding and progresses through four stages, each containing multiple ConvNeXt Blocks operating at progressively lower spatial resolutions and higher channel dimensions. Each ConvNeXt block contains a  $7 \times 7$  depthwise convolution, followed by layer normalization and pointwise convolution. This is then fed to a GELU activation function into a final pointwise convolution with the same output dimension as the original input. Each block contains a residual connection to stabilize training and preserve information. The purpose is for the model to be able to capture both local and global features within the image, without making use of any attention mechanisms.

### IV. TRAINING

In this section, we describe the training strategy used for geographic localization. Outlining how S2 cells are incorporated into the classification framework, and detailing the loss formulation that enables learning across multiple spatial resolutions.

#### A. Loss and S2

To model geographic location as a classification problem, we discretize the Earth's surface using the S2 geometry library[4]. S2 partitions the sphere into a hierarchical grid of cells: at low levels the cells are large and cover coarse regions (e.g. continents or countries), while higher levels correspond to smaller, more fine-grained geographic areas. For each panorama in the dataset, we know its latitude and longitude. During preprocessing, we map each coordinate to S2 cell IDs at several resolutions (e.g., levels  $L_3 \dots L_6$ ), and then convert these S2 IDs into contiguous class indices via lookup tables. This gives us a hierarchical label vector for each image, where each element corresponds to the region at a different spatial scale. We also precompute this relationship as a set of parent maps, where a lookup such as

$$\text{Parents}[(6, 5)][i] = j$$

tells us that the fine-level class index  $i$  at level 6 lies inside the coarse-level class index  $j$  at level 5. We use this hierarchical labeling to implement a hierarchical cross-entropy loss. This approach supervises the model at multiple spatial resolutions (S2 levels) simultaneously: the model outputs class probabilities at the finest level, we aggregate these probabilities to coarser cells via the S2 parent maps, and apply a weighted CE loss at each level. This should encourage the network to first get the broad region roughly correct and then refine its prediction within that area. In turn, this makes training faster, more stable, and more sample efficient. Effectively treating nearby cells as more similar than distant ones, which naturally reflects the actual world. Ideally, the loss would be defined directly in terms of how visually similar two regions (or an image and an area) are, but our hierarchical loss is a practical proxy that attempts to partially capture this intuition. That is, the total loss is defined as,

$$L = \sum_L w_L H(y^{(L)}, p^{(L)})$$

Where  $H$  is the cross-entropy loss  $y^{(L)}$  is the ground-truth class label at S2 level  $L$ , and  $p^{(L)}$  is the corresponding predicted class distribution at that level, and  $w_L$  is user-defined weights (hyperparameter) that controls the relative contribution of level  $L$  (we used;  $w_L = [0.4, 0.6, 0.8, 1.0, 1.2]$ ). In our implementation, we apply label smoothing with a parameter value of  $\epsilon = 0.1$ , to the cross-entropy, which slightly relaxes the one-hot encoded targets and can improve generalization on large-scale classification problems.

### V. HYPERPARAMETER TUNING

As with most deep learning tasks, image classification involves tuning a large number of hyperparameters. However, given the large dataset and the long training time, a complete optimization was not feasible. Therefore for any hyperparameters not discussed here, it was decided to use the recommended values reported in [2].

Although large datasets typically use batch sizes of 128 or 256, due to GPU memory limitations, we reduced the batch size to 64. We also experimented with different approaches to learning rates. Initially, we attempted to use a fixed learning rate of  $l_r = 5e - 5$ , but the training process was significantly improved by replacing the fixed value with the CosineAnnealingLR[5] learning rate scheduler. Training was limited to  $\text{max\_epochs} = 64$  and was done using the ADAM optimizer.

### VI. RESULTS

For this section, we will go over the results achieved for the models general performance.

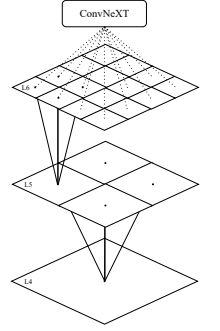


Fig. 3: Visualization of S2 cells layer structure

### A. Training Loss

Figure 4 shows the training loss for the  $L_6$  model. The training loss decreases rapidly within the first few epochs, indicating that the model is able to quickly learn useful representations from the large dataset. After this initial drop, the curve continues to decline more gradually, suggesting incremental improvement as training progresses.

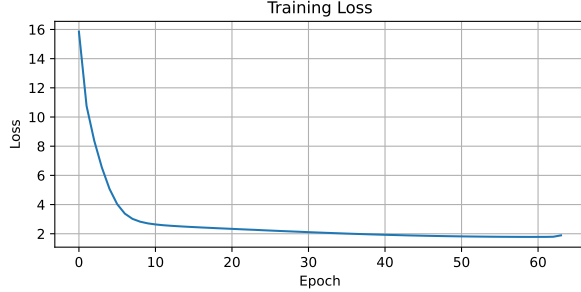


Fig. 4: Training loss achieved during training over 64 epochs.

Although omitting validation loss was an oversight (and could not be corrected due to training time constraints), this limitation is unlikely to affect the conclusions made. An intermediate checkpoint saved at 30 epochs showed lower performance than the final model at 64 epochs (on hold-out test set), and the dataset size is sufficiently large relative to the model, that overfitting likely did not occur.

### B. Geodesic Error

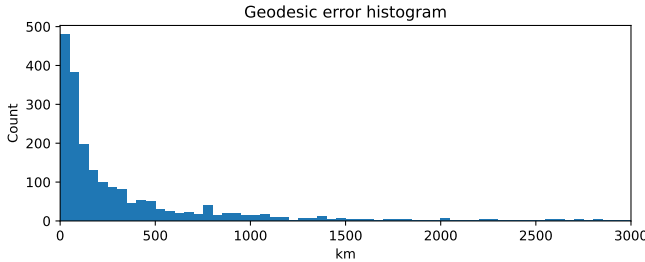


Fig. 5: Histograms showing the distribution of geodesic errors. Bin width: 50 km. Range limited to 3000 km, occasional larger errors occurred beyond this limit.

From figure 5, it can be seen that the majority of test samples have errors that fall within the lower range (0 - 200 km), indicating that the method performs reliably in most cases. The distribution is heavily right-skewed, with a long tail extending toward higher error values. This suggests that although typical performance is strong, a smaller number of instances produce substantially larger deviations, likely due to difficult input imagery with few identifiable features, data gaps in the area, or instability from sparse input.

### C. Accuracy

The results in Table II show a consistent trend across S2 levels and Top-k settings. For all Top-k values, accuracy decreases

| Top-k | S2 level | Accuracy | Random baseline |
|-------|----------|----------|-----------------|
| 1     | 3        | 0.7130   | 0.0058          |
| 1     | 4        | 0.6130   | 0.0022          |
| 1     | 5        | 0.4730   | 0.0008          |
| 1     | 6        | 0.3380   | 0.0003          |
| 1     | 7        | 0.2320   | 0.0001          |
| 5     | 3        | 0.9255   | 0.0289          |
| 5     | 4        | 0.9090   | 0.0111          |
| 5     | 5        | 0.8185   | 0.0039          |
| 5     | 6        | 0.6575   | 0.0013          |
| 5     | 7        | 0.4875   | 0.0005          |
| 10    | 3        | 0.9495   | 0.0578          |
| 10    | 4        | 0.9430   | 0.0222          |
| 10    | 5        | 0.8845   | 0.0079          |
| 10    | 6        | 0.7445   | 0.0027          |
| 10    | 7        | 0.5900   | 0.0009          |

TABLE II: Test accuracy and random baseline by Top-k and S2 level. On separate Test dataset  $\approx 2000$  images.

as the S2 level increases, reflecting the greater difficulty of making precise predictions at finer spatial resolutions. However, even at higher S2 levels, the model performs far above the random baseline, indicating that it is learning meaningful geographic signals rather than relying on chance.

Increasing Top-k substantially improves performance. Moving from Top 1 to Top 5 picks yields a large accuracy gain across all S2 levels, and Top 10 provides an additional improvement, although with diminishing returns. At coarse resolutions (e.g., S2 level 3), accuracy approaches 95% for Top 10 retrieval, while for finer resolutions (S2 level 7), performance remains considerably higher than the random baseline despite increased spatial granularity.

Overall, the results demonstrate that the model is highly effective at coarse to moderate resolutions and maintains strong relative performance even as the prediction task becomes more spatially specific.

## VII. LIMITATIONS & IMPROVEMENTS

Training time was a significant bottleneck, with convergence taking approximately  $\sim 90$  hours. This constrained our ability to conduct a broader hyperparameter search and limited the use of higher resolution images for training. It was also necessary to use a smaller model architecture (ConvNeXT-Base) and limit the S2 level range.

## REFERENCES

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- [2] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s, 2022. URL <https://arxiv.org/abs/2201.03545>.
- [3] GeoGuessr Community. Catalog of geoguessr maps. <https://docs.google.com/spreadsheets/d/1qXtRSZolj8RRzCib6Im7Cuvx4FumYTXT1SYDAIcu8W4/>, 2024. Community-compiled empirical distribution data.
- [4] Jonathan Gillham and contributors. s2sphere (version 0.2.5), 2017. URL <https://pypi.org/project/s2sphere/>.
- [5] PyTorch Contributors. Cosineannealinglr. [https://docs.pytorch.org/docs/stable/generated/torch.optim.lr\\_scheduler.CosineAnnealingLR.html](https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html), 2025. Pytorch learning rate scheduler.